

OSY Manual Solution

Practical No. 1: System call commands in Linux such as fork(), exec(), getpid, pipe, exit, open, close, stat, uname.

Conclusion : Successfully implemented System call commands in Linux such as fork(), exec(), getpid, pipe, exit, open, close, stat, uname.

Practical related questions

1. Write use of system calls.

Answer: System calls provide an interface between user programs and the operating system kernel. They allow programs to perform tasks such as file handling, process creation, memory management, and communication with hardware.

2. Write system calls related to Process Management.

Answer:

- fork()
- exec()
- getpid()
- exit()
- wait()

3. Write a command which returns a PID.

Answer: getpid()

4. What are various options for uname?

Answer:

- -a : Display all information
- -s : Kernel name
- -n : Network node name
- -r : Kernel release
- -v : Kernel version
- -m : Machine hardware
- -p : Processor type
- -i : Hardware platform
- -o : Operating system

5. Explain about exit command.

Answer: The exit() system call terminates the execution of a process. After termination, the operating system releases all resources allocated to that process.

6. List any four system calls and write their use.

Answer:

System Call	Use
fork()	Creates a new process
exec()	Executes a new program
open()	Opens a file
close()	Closes an opened file

Practical No. 2: Process related commands in Linux-top, ps, kill, wait, sleep, nice, renice, bg, fg.

Conclusion : Successfully implemented Process related commands in Linux-top, ps, kill, wait, sleep, nice, renice, bg, fg.

Practical related questions

1. What is the difference between sleep and wait?

Answer:

sleep	wait
Pauses execution for a specified time.	Waits until a child process finishes.

2. Write various signal names along with their signal number used in kill command.

Answer:

Signal	Number
SIGHUP	1
SIGINT	2
SIGQUIT	3
SIGKILL	9
SIGTERM	15

3. What is the name of your login shell?

Answer: Bash (bash) (*commonly used in Linux systems*).

4. List the system calls for process management.

Answer:

- fork()
- exec()
- wait()
- exit()
- getpid()

5. Explain nice and renice command.

Answer:

- **nice:** Starts a new process with a specified priority.
- **renice:** Changes the priority of an already running process.

6. What are various options of ps command?

Answer:

- ps -f
- ps -u username
- ps -a
- ps -e

7. What is process ID of your login shell?

Answer: It depends on the running system.

It can be obtained using:

echo \$\$

Practical No. 3:

a. Commands for Sending Messages to Logged-In Use who, cat, wall, write, mesg.

b. List Processes Attached to a Shared Memory Segment: ipcs.

Conclusion : Successfully implemented :

a. Commands for Sending Messages to Logged-In Use who, cat, wall, write, mesg.

b. List Processes Attached to a Shared Memory Segment: ipcs.

Practical related questions

1. What are the prerequisites for using write command?

Answer:

- Both users must be logged into the system.
- The receiver must allow messages (mesg y).

2. Write a command to display the current run level using who command.

Answer: who -r

3. Write a command to append the contents of one file to another using cat command.

Answer: cat file1 >> file2

4. Write a command to send a message from a file using wall command.

Answer: wall < message.txt

5. Write a command to check the current message permission status of your terminal.

Answer: mesg

6. What is the purpose of the ipcs command in Linux?

Answer: The ipcs command displays information about Inter-Process Communication (IPC) resources such as shared memory segments, message queues, and semaphore sets.

Practical No. 4: Write C/Python program to calculate average waiting time and turnaround time of n processes with First Come First Serve (FCFS) CPU scheduling algorithm.

Conclusion : Successfully implemented C/Python program to calculate average waiting time and turnaround time of n processes with First Come First Serve (FCFS) CPU scheduling algorithm.

Practical related questions

1. State the conditions for preemptive and non-preemptive scheduling algorithm.

Answer:

- **Preemptive:** The CPU can be taken away from a running process before it completes.
- **Non-preemptive:** Once a process gets the CPU, it continues until completion or it enters the waiting state.

2. State the disadvantages of FCFS.

Answer:

- Poor average waiting time.
- Convoy effect.
- Short processes may wait behind long processes.
- Not suitable for time-sharing systems.

3. Define Turnaround Time. Write the formula for average Turnaround Time.

Answer:

Turnaround Time (TAT): Time from process arrival to completion.

Formula:

$TAT = \text{Completion Time} - \text{Arrival Time}$

Average Turnaround Time:

$\text{Average TAT} = \text{Sum of all TAT} / \text{Number of Processes}$

4. Define Waiting Time. Write the formula for average Waiting Time.

Answer:

Waiting Time (WT): Time a process spends waiting in the ready queue.

Formula:

$WT = \text{Turnaround Time} - \text{Burst Time}$

Average Waiting Time:

$\text{Average WT} = \text{Sum of all WT} / \text{Number of Processes}$

5. Solve using FCFS.

Given:

Process	Arrival Time	Burst Time
P0	0	10
P1	1	4
P2	2	14
P3	3	8

Gantt Chart

0 10 14 28 36
| P0 | P1 | P2 | P3 |

Process	CT	TAT	WT
P0	10	10	0
P1	14	13	9
P2	28	26	12
P3	36	33	25

Average Turnaround Time = $(10 + 13 + 26 + 33) / 4 = 20.5$ ms

Average Waiting Time = $(0 + 9 + 12 + 25) / 4 = 11.5$ ms

Practical No. 5: Write C/Python program to calculate average waiting time and Turnaround Time of n processes with Shortest Job First(SJF) CPU scheduling algorithm.

Conclusion : Successfully implemented C/Python program to calculate average waiting time and Turnaround Time of n processes with Shortest Job First(SJF) CPU scheduling algorithm.

Practical related questions

1. Explain the difference between preemptive and non-preemptive SJF.

Answer:

Feature	Non-Preemptive SJF	Preemptive SJF (Shortest Remaining Time First - SRTF)
Can interrupt a running process?	No	Yes
CPU selection	Chooses the shortest job and runs it until completion	Always runs the process with the shortest remaining execution time
Better response time	No	Yes
Context switches	Fewer	More

2. What is the main advantage of SJF scheduling?

Answer:

The **main advantage of Shortest Job First (SJF) scheduling** is that it **minimizes the average waiting time** of processes compared to many other CPU scheduling algorithms.

- Minimum average waiting time
- Better CPU utilization
- Higher throughput (more processes completed in a given time)
- Short jobs receive faster service, improving response time for those tasks

3. State the disadvantages of SJF Scheduling.

Answer:

- Requires prior knowledge or estimation of CPU burst time.
- Long processes may suffer from starvation.
- Burst time estimation is difficult and may be inaccurate.
- Not suitable for interactive or real-time systems due to possible long waiting times for larger processes.

4. Consider the processes P1,P2 ,P3,P4 given in the below table. Find average waiting time and average turnaround time by using SJF(non-preemptive and preemptive) Scheduling algorithm.

Process	Burst Time	Arrival Time
P1	6	0
P2	3	1
P3	4	2
P4	10	3

Answer:

Given:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	6
P2	1	3
P3	2	4
P4	3	10

1. SJF (Non-Preemptive)

Step 1: Draw Gantt Chart

- At $t = 0$, only **P1** has arrived, so it starts.
- P1 runs until completion.

At $t = 6$, available processes are:

- P2 (BT = 3)
- P3 (BT = 4)
- P4 (BT = 10)

Choose the shortest → **P2**

Then P3, then P4.

Gantt Chart

0 6 9 13 23
 | P1 | P2 | P3 | P4 |

Step 2: Completion Time (CT)

Process	AT	BT	CT
P1	0	6	6
P2	1	3	9
P3	2	4	13
P4	3	10	23

Step 3: Turnaround Time

$$TAT = CT - AT$$

Process	CT	AT	TAT
P1	6	0	6
P2	9	1	8
P3	13	2	11
P4	23	3	20

Step 4: Waiting Time

$$WT = TAT - BT$$

Process	TAT	BT	WT
P1	6	6	0
P2	8	3	5
P3	11	4	7
P4	20	10	10

Average Waiting Time

$$\frac{0 + 5 + 7 + 10}{4} = \frac{22}{4} = 5.5$$

Average Turnaround Time

$$\frac{6 + 8 + 11 + 20}{4} = \frac{45}{4} = 11.25$$

Non-Preemptive SJF Results

- Average Waiting Time = 5.5 units
 - Average Turnaround Time = 11.25 units
-

2. SJF (Preemptive / Shortest Remaining Time First)

Step-by-Step Scheduling

- **0–1:** P1 runs (remaining = 5)
- **1:** P2 arrives (BT = 3 < 5), so P1 is preempted.
- **1–4:** P2 completes.
- **4:** Available:
 - P1 = 5
 - P3 = 4
 - P4 = 10

Choose **P3**.

- **4–8:** P3 completes.
- **8–13:** P1 completes.
- **13–23:** P4 completes.

Gantt Chart

0 1 4 8 13 23
| P1 | P2 | P3 | P1 | P4 |

Completion Time

Process	CT
P1	13
P2	4
P3	8
P4	23

Turnaround Time

TAT = CT – AT

Process	CT	AT	TAT
P1	13	0	13
P2	4	1	3
P3	8	2	6
P4	23	3	20

Waiting Time

WT = TAT – BT

Process	TAT	BT	WT
P1	13	6	7
P2	3	3	0
P3	6	4	2
P4	20	10	10

Average Waiting Time

$$\frac{7 + 0 + 2 + 10}{4} = \frac{19}{4} = 4.75$$

Average Turnaround Time

$$\frac{13 + 3 + 6 + 20}{4} = \frac{42}{4} = 10.5$$

Preemptive SJF (SRTF) Results

- **Average Waiting Time = 4.75 units**
 - **Average Turnaround Time = 10.5 units**
-

Practical No. 6: Write C/Python program to calculate average waiting time and Turnaround Time of n processes with Priority CPU scheduling algorithm.

Conclusion : Successfully implemented C/Python program to calculate average waiting time and Turnaround Time of n processes with Priority CPU scheduling algorithm.

Practical related questions

1. What is priority CPU scheduling?

Answer:

Priority CPU Scheduling is a CPU scheduling algorithm in which each process is assigned a **priority**, and the CPU is allocated to the process with the **highest priority** (or the lowest priority number, depending on the system).

- If two processes have the same priority, they are usually scheduled using **First Come First Served (FCFS)**.
- Priority scheduling can be **preemptive** or **non-preemptive**.

2. How is the priority of a process determined in a priority scheduling algorithm?

Answer:

The **priority of a process** in a priority scheduling algorithm can be determined in two ways:

1. Static Priority

- The priority is assigned when the process is created and **does not change** during its execution.
- Example: System processes may always have higher priority than user processes.

2. Dynamic Priority

- The priority can **change over time** based on factors such as:
 - Waiting time (to prevent starvation)
 - CPU usage
 - Process behaviour
 - Deadlines or urgency
- Example: A process waiting for a long time may have its priority increased through **aging**.

3. What problem can arise in priority scheduling and how can it be resolved?

Answer:

Problem: The main problem in priority scheduling is **starvation**, where low-priority processes may wait indefinitely because higher-priority processes keep arriving.

Solution: This problem is resolved using **aging**, which gradually increases the priority of waiting processes so that they eventually receive CPU time.

4. Consider the processes P1,P2 ,P3 given in the below table. Find average waiting time and average turnaround time by using priority (non-preemptive and preemptive) Scheduling algorithm.

Process	Burst Time	Arrival Time	Priority
P1	6	0	2
P2	4	1	3
P3	5	2	1

Answer:

Assume **smaller priority number = higher priority**.

Given Data

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	6	4
P2	1	2	1
P3	2	5	2

1. Priority Scheduling (Non-Preemptive)

Step 1: Gantt Chart

- At **t = 0**, only P1 is available → execute P1.
- P1 finishes at **t = 6**.
- Available: P2 (Priority 1), P3 (Priority 2).
- Highest priority is P2, then P3.

Gantt Chart

0 6 8 13
 | P1 | P2 | P3 |

Step 2: Completion Time (CT)

Process	CT
P1	6
P2	8
P3	13

Step 3: Turnaround Time (TAT)

TAT = CT – AT

Process	CT	AT	TAT
P1	6	0	6
P2	8	1	7
P3	13	2	11

Step 4: Waiting Time (WT)

$$WT = TAT - BT$$

Process	TAT	BT	WT
P1	6	6	0
P2	7	2	5
P3	11	5	6

Average Waiting Time

$$\frac{0 + 5 + 6}{3} = \frac{11}{3} = 3.67$$

Average Turnaround Time

$$\frac{6 + 7 + 11}{3} = \frac{24}{3} = 8$$

Non-Preemptive Results

- Average Waiting Time = 3.67 units
- Average Turnaround Time = 8 units

2. Priority Scheduling (Preemptive)

Step-by-Step Execution

- **0–1:** P1 runs.
- **t = 1:** P2 arrives with higher priority ($1 < 4$), so P1 is preempted.
- **1–3:** P2 completes.
- **t = 3:** P3 (Priority 2) has higher priority than remaining P1 (Priority 4).
- **3–8:** P3 completes.
- **8–13:** P1 completes.

Gantt Chart

0 1 3 8 13
| P1 | P2 | P3 | P1 |

Completion Time (CT)

Process	CT
P1	13
P2	3
P3	8

Turnaround Time (TAT)

Process	CT	AT	TAT
P1	13	0	13
P2	3	1	2
P3	8	2	6

Waiting Time (WT)

Process	TAT	BT	WT
P1	13	6	7
P2	2	2	0
P3	6	5	1

Average Waiting Time

$$\frac{7 + 0 + 1}{3} = \frac{8}{3} = 2.67$$

Average Turnaround Time

$$\frac{13 + 2 + 6}{3} = \frac{21}{3} = 7$$

Preemptive Results

- Average Waiting Time = 2.67 units
 - Average Turnaround Time = 7 units
-

Practical No. 7: Write C/Python program to calculate average waiting time and Turnaround Time of n processes with Round Robin (RR) CPU scheduling algorithm.

Conclusion : Successfully implemented C/Python program to calculate average waiting time and Turnaround Time of n processes with Round Robin (RR) CPU scheduling algorithm.

Practical related questions**1. What is time quantum?**

Answer : Time quantum (or time slice) is the fixed amount of CPU time assigned to a process in Round Robin scheduling. If the process does not complete within this time, it is preempted and moved to the end of the ready queue, allowing the next process to execute.

2. State advantages and disadvantages of round robin scheduling.

Answer :

Advantages:

- Fair CPU allocation to all processes.
- Prevents starvation.
- Provides good response time for interactive systems.
- Easy to implement.

Disadvantages:

- High context-switching overhead.
- Performance depends on the choice of time quantum.
- Can result in higher waiting and turnaround times.
- Less efficient for long CPU-bound processes.

3. How context switching occurs in Round Robin scheduling and its impact on system performance?

Answer :

1. A process is assigned the CPU for one time quantum.
2. If the process finishes within that time, the CPU is allocated to the next process.
3. If the process does not finish, the operating system:
 - Saves the current process's state (registers, program counter, etc.).
 - Moves the process to the end of the ready queue.
 - Loads the state of the next process.
4. The CPU then starts executing the next process.

4. Consider the processes P1,P2 ,P3,P4,p5 given in the below table. Find average waiting time and average turnaround time by using Round Robin Scheduling algorithm. Time quantum=3ms

Process	Burst Time	Arrival Time
P1	8	0
P2	7	1
P3	2	5
P4	3	6
P5	5	8

Answer :

Given

Time Quantum (TQ) = 3 ms

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	8
P2	1	7
P3	2	2
P4	3	5
P5	5	6

Step 1: Gantt Chart

0 3 6 8 11 14 17 20 22 25 27 28
 | P1 | P2 | P3 | P4 | P1 | P5 | P2 | P4 | P1 | P5 | P2 |

Step 2: Completion Time (CT)

Process	CT
P1	25
P2	28
P3	8
P4	22
P5	27

Step 3: Turnaround Time (TAT)

$$TAT = CT - AT$$

Process	CT	AT	TAT
P1	25	0	25
P2	28	1	27
P3	8	2	6
P4	22	3	19
P5	27	5	22

Step 4: Waiting Time (WT)

$$WT = TAT - BT$$

Process	TAT	BT	WT
P1	25	8	17
P2	27	7	20
P3	6	2	4
P4	19	5	14
P5	22	6	16

Step 5: Average Waiting Time

$$\frac{17 + 20 + 4 + 14 + 16}{5} = \frac{71}{5} = 14.2 \text{ ms}$$

Step 6: Average Turnaround Time

$$\frac{25 + 27 + 6 + 19 + 22}{5} = \frac{99}{5} = 19.8 \text{ ms}$$

Final Answer

Process	AT	BT	CT	TAT	WT
P1	0	8	25	25	17
P2	1	7	28	27	20
P3	2	2	8	6	4
P4	3	5	22	19	14
P5	5	6	27	22	16

- Average Waiting Time = 14.2 ms
 - Average Turnaround Time = 19.8 ms
-

Practical No. 8: Write a C/Python program to implement Banker's Algorithm.

Conclusion : Successfully implemented a C/Python program to implement Banker's Algorithm.

Practical related questions

1. What is meant by deadlock ?

Answer :

A deadlock is a situation in an operating system where two or more processes are permanently blocked because each process is waiting for a resource that is being held by another process. As a result, none of the processes can continue executing.

Example

- Process P1 holds Resource R1 and is waiting for Resource R2.
- Process P2 holds Resource R2 and is waiting for Resource R1.

Both processes wait forever, causing a deadlock.

2. What is safe state in banker's algorithms?

Answer :

A **safe state** is a state in which the system can allocate resources to all processes in some order (called a **safe sequence**) so that every process can complete its execution without causing a deadlock.

- In a safe state, the operating system can satisfy the resource needs of all processes one after another.
- A safe state **guarantees that deadlock will not occur** if the processes follow the safe sequence.

Example

If the safe sequence is:

P2 → P1 → P3 → P4

then each process can obtain the required resources, complete its execution, release its resources, and allow the next process to proceed.

3. What is banker's algorithm?

Answer :

Banker's Algorithm is a **deadlock avoidance algorithm** used by the operating system to allocate resources safely. Before granting a resource request, it checks whether the system will remain in a **safe state**. If granting the request would lead to an unsafe state, the request is postponed.

4. What happens if a resource request leads to an unsafe state?

Answer :

If a resource request leads to an unsafe state, the Banker's Algorithm denies or postpones the request. The process is kept waiting until the resources can be allocated while keeping the system in a safe state, thereby avoiding deadlock.

5. What are the necessary conditions where deadlock occurs?

Answer :

A deadlock can occur only if **all four** of the following conditions are present:

1. Mutual Exclusion

- At least one resource is non-shareable, meaning only one process can use it at a time.

2. Hold and Wait

- A process holds one or more resources while waiting for additional resources held by other processes.

3. No Preemption

- Resources cannot be forcibly taken from a process; they must be released voluntarily after the process completes.

4. Circular Wait

- A circular chain of processes exists where each process is waiting for a resource held by the next process in the chain.
-

Practical No. 9: Basic memory management commands - df, free, vmstat, /proc/meminfo, htop

Conclusion : Successfully implemented Basic memory management commands - df, free, vmstat, /proc/meminfo, htop.

Practical related questions

1. What is memory management in an operating system?

Answer :

Memory management is the process by which the operating system manages the computer's main memory by allocating and deallocating memory to processes, keeping track of memory usage, and ensuring efficient and secure utilization of memory.

2. How can you check the total amount of RAM on a Linux system using a command?

Answer :

1. Using free (most common)

`free -h`

- Displays total, used, free, and available memory in a human-readable format (KB, MB, GB).

2. Using /proc/meminfo

`cat /proc/meminfo`

- Shows detailed memory information. The **MemTotal** field indicates the total RAM.

3. Using vmstat

`vmstat -s`

- Displays memory statistics, including the total memory.

3. What does the vmstat command show, and what are its key fields?

Answer :

The **vmstat (Virtual Memory Statistics)** command displays information about **system performance**, including memory usage, processes, CPU activity, paging, disk I/O, and system interrupts.

Syntax

`vmstat`

or

`vmstat -s`

Its key fields include r, b, swpd, free, buff, cache, si, so, bi, bo, in, cs, us, sy, id, and wa.

4. Which commands provides the current memory usage details?

Answer :

Commands that provide current memory usage details are:

- `free -h` – Shows total, used, free, and available memory.
- `vmstat` – Displays memory, CPU, and I/O statistics.
- `cat /proc/meminfo` – Shows detailed memory information.
- `top` / `htop` – Provides real-time memory and process usage.

5. Write the work of htop command.

Answer :

The `htop` command is an interactive Linux utility used to monitor system performance. It displays real-time information about CPU usage, memory usage, swap usage, and running processes, and allows users to manage processes by searching, sorting, and terminating them easily.

6. How do you monitor memory usage trends over time?

Answer :

1. `vmstat`

`vmstat 2`

- Displays memory, CPU, and I/O statistics every **2 seconds**.

2. `free`

`watch -n 2 free -h`

- Updates the memory usage display every **2 seconds**.

3. `top` or `htop`

`top`

or

`htop`

- Continuously displays real-time memory usage and running processes.

4. `sar` (from the `sysstat` package)

`sar -r 2 10`

- Reports memory usage every **2 seconds** for **10 intervals**, making it useful for analyzing memory trends over time.
-

Practical No. 10: Write a C/Python program on First In First Out (FIFO) Page Replacement algorithm.

Conclusion : Successfully implemented a C/Python program on First In First Out (FIFO) Page Replacement algorithm.

Practical related questions

1. What is page hit ?

Answer :

A **page hit** occurs when the page requested by the CPU is **already present in the main memory (RAM)**. Since the required page is available, it can be accessed immediately without loading it from secondary storage (disk).

Example

If **Page 5** is already in RAM and the CPU requests **Page 5**, it is a **page hit**.

2. What is page fault?

Answer :

A **page fault** occurs when the page requested by the CPU is **not present in the main memory (RAM)**. The operating system must fetch the required page from **secondary storage (disk)** into RAM before the process can continue execution.

Example

If the CPU requests **Page 7**, but **Page 7** is not in RAM, the operating system loads it from disk into RAM. This is called a **page fault**.

3. What is FIFO page replacement? How does it work?

Answer :

FIFO Page Replacement is a page replacement algorithm in which the **page that has been in memory the longest (the first page loaded)** is replaced when a new page needs to be loaded and all memory frames are full.

How It Works

1. Pages are loaded into memory in the order they arrive.
2. When a page fault occurs and all frames are full:
 - The **oldest page** (the one loaded first) is removed.
 - The new page is loaded into the freed frame.
3. This process continues using the **First-In, First-Out (FIFO)** rule.

Example

Suppose there are **3 page frames** and the page reference string is:

1, 2, 3, 4

- Load **1**, **2**, and **3** into the three frames.
- When **4** is requested, the frames are full.
- **Page 1** is the oldest page, so it is removed and replaced with **4**.

4. State the advantages and disadvantages of FIFO.

Answer :

Advantages:

- Simple and easy to implement.
- Low overhead.
- Uses a straightforward queue mechanism.
- Fairly replaces the oldest page first.

Disadvantages:

- May replace frequently used pages.
- Suffers from **Belady's Anomaly** (Increasing the number of page frames may increase the number of page faults).
- Does not consider page usage or recency.
- Can result in a higher number of page faults than other algorithms.

5. Count number of page hits in above mention example given in section IX.

Answer :

Given:

- **Reference String:** 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 4, 5, 6, 7
- **Frame Size:** 3
- **Page Replacement Algorithm:** FIFO

Reference	Frames After Access	Hit/Fault
0	0	Fault
1	0 1	Fault
2	0 1 2	Fault
3	1 2 3	Fault
0	2 3 0	Fault
1	3 0 1	Fault
2	0 1 2	Fault
3	1 2 3	Fault
0	2 3 0	Fault
1	3 0 1	Fault
2	0 1 2	Fault
3	1 2 3	Fault
4	2 3 4	Fault
5	3 4 5	Fault
6	4 5 6	Fault
7	5 6 7	Fault

Count

- Page Hits = 0
- Page Faults = 16

Reason: With only **3 frames** and the repeating pattern **0, 1, 2, 3**, each page is replaced before it is referenced again. Therefore, every reference results in a **page fault**, and there are **no page hits**.

6. Given the following page reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 How many page faults will occur using FIFO with 3 frames?

Answer :

Given

- Page Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Frame Size: 3
- Page Replacement Algorithm: FIFO

Step-by-Step Solution

Reference	Frames After Access	Hit/Fault
1	1	Fault
2	1 2	Fault
3	1 2 3	Fault
4	2 3 4	Fault
1	3 4 1	Fault
2	4 1 2	Fault
5	1 2 5	Fault
1	1 2 5	Hit
2	1 2 5	Hit
3	2 5 3	Fault
4	5 3 4	Fault
5	5 3 4	Hit

Count

- Page Faults = 9
- Page Hits = 3

Final Answer

- Number of Page Faults = 9
 - Number of Page Hits = 3
-

Practical No. 11: Write a C/Python program on Least Recently Used (LRU) Page Replacement algorithm.

Conclusion : Successfully implemented C/Python program on Least Recently Used (LRU) Page Replacement algorithm.

Practical related questions**1. Compare FIFO and LRU.**

Answer :

FIFO (First-In, First-Out)	LRU (Least Recently Used)
Replaces the oldest page in memory.	Replaces the least recently used page .
Does not consider how recently a page was used.	Considers recent page usage before replacement.
Simple and easy to implement.	More complex to implement.
Lower implementation overhead.	Higher overhead due to tracking page usage.
Can suffer from Belady's Anomaly .	Does not suffer from Belady's Anomaly.
Usually results in more page faults.	Usually results in fewer page faults and better performance.

2. State the advantages and disadvantages of LRU.

Answer :

Advantages of LRU

1. **Reduces page faults** compared to FIFO in most cases.
2. **Considers recent page usage**, so frequently used pages are less likely to be replaced.
3. **Does not suffer from Belady's Anomaly**.
4. **Provides better overall performance** than FIFO for many workloads.

Disadvantages of LRU

1. **More complex to implement** than FIFO.
2. **Requires tracking page usage**, which increases overhead.
3. **Needs extra memory or hardware support** (such as counters or stacks) to keep track of recently used pages.
4. **Higher implementation cost** due to maintaining usage history.

3. Count number of page hits and page faults in above mention example given in section IX.

Answer :

Given

- Page Reference String: 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 4, 5, 6, 7
- Frame Size: 3
- Page Replacement Algorithm: LRU (Least Recently Used)

Step-by-Step

Reference	Frames After Access	Hit/Fault
0	0	Fault
1	0 1	Fault
2	0 1 2	Fault
3	1 2 3	Fault
0	2 3 0	Fault
1	3 0 1	Fault
2	0 1 2	Fault
3	1 2 3	Fault
0	2 3 0	Fault
1	3 0 1	Fault
2	0 1 2	Fault
3	1 2 3	Fault
4	2 3 4	Fault
5	3 4 5	Fault
6	4 5 6	Fault
7	5 6 7	Fault

Count

- Page Hits = 0
- Page Faults = 16

Final Answer

- Number of Page Hits = 0
- Number of Page Faults = 16

Reason: With only **3 page frames** and the repeating sequence **0, 1, 2, 3**, each page is replaced before it is referenced again. Therefore, under **LRU**, every page reference results in a **page fault**, and there are **no page hits**.

4. You are given the following page reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
Using 3 page frames, how many page faults occur with LRU?

Answer :

Given

- Page Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Page Frames: 3
- Page Replacement Algorithm: LRU (Least Recently Used)

Step-by-Step Solution

Reference	Frames After Access	Hit/Fault
1	1	Fault
2	1 2	Fault
3	1 2 3	Fault
4	2 3 4	Fault
1	3 4 1	Fault
2	4 1 2	Fault
5	1 2 5	Fault
1	1 2 5	Hit
2	1 2 5	Hit
3	1 2 3	Fault
4	2 3 4	Fault
5	3 4 5	Fault

Count

- Page Faults = 10
- Page Hits = 2

Final Answer

- Number of Page Faults = 10
 - Number of Page Hits = 2
-

Practical No. 12: Write a C/Python program on sequential file allocation method.

Conclusion : Successfully implemented a C/Python program on sequential file allocation method.

Practical related questions**1. What are the various file allocation strategies?**

Answer :

The three main file allocation strategies are:

1. **Contiguous Allocation** – Stores files in consecutive disk blocks.
2. **Linked Allocation** – Stores file blocks at different locations connected by pointers.
3. **Indexed Allocation** – Uses an index block to store the addresses of all file blocks.

2. What is Sequential file Allocation Method?

Answer :

Sequential (Contiguous) File Allocation is a method in which a file is stored in consecutive disk blocks. The operating system records the starting block and the file length, providing fast access but causing external fragmentation and making file expansion difficult.

Example

If a file requires **4 blocks**, it may be stored in blocks:

10 → 11 → 12 → 13

3. What are the disadvantages of sequential allocation methods?

Answer :

1. **External Fragmentation**
 - Free disk space becomes scattered into small blocks, making it difficult to allocate large contiguous space for new files.
2. **Difficult File Expansion**
 - If a file grows in size, there may not be enough contiguous free space available next to it.
3. **Space Wastage**
 - Extra space may be reserved for future file growth, leading to unused disk space.
4. **File Relocation**
 - If contiguous space is unavailable, the entire file may need to be moved to another location, which is time-consuming.

4. Compare sequential allocation with linked and indexed file allocation methods.

Answer :

Sequential (Contiguous) Allocation	Linked Allocation	Indexed Allocation
Stores files in consecutive disk blocks .	Stores file blocks anywhere on the disk and links them using pointers.	Uses a separate index block that stores the addresses of all file blocks.
Supports fast sequential and direct access .	Good for sequential access but slow for direct/random access.	Supports both sequential and direct/random access efficiently.
Causes external fragmentation .	No external fragmentation.	No external fragmentation.
File expansion is difficult .	File expansion is easy.	File expansion is easy by updating the index block.
Simple to implement.	Moderate complexity.	More complex because of index management.
No pointer overhead.	Requires pointer space in each block.	Requires extra space for the index block.

5. Compare sequential allocation with linked and indexed file allocation methods.

Answer :

Sequential (Contiguous) Allocation	Linked Allocation	Indexed Allocation
Stores files in consecutive disk blocks .	Stores file blocks anywhere on the disk and links them using pointers.	Uses a separate index block that stores the addresses of all file blocks.
Supports fast sequential and direct access .	Good for sequential access but slow for direct/random access.	Supports both sequential and direct/random access efficiently.
Causes external fragmentation .	No external fragmentation.	No external fragmentation.
File expansion is difficult .	File expansion is easy.	File expansion is easy by updating the index block.
Simple to implement.	Moderate complexity.	More complex because of index management.
No pointer overhead.	Requires pointer space in each block.	Requires extra space for the index block.
